Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică
Program: Software Engineering

Operating Systems

Laboratory Work No. 8

The Same Calls, Inside Your Own OS

Track B: Build Your Own Small OS (xv6)

Elaborated by:

st. gr. FAF-241
Luchiciov Alexei

Verified by:

Reiţman Patricia

Chișinău – 2026

Contents

1. Introduction 3
2. Preparation and xv6 Build 3
2.1. Makefile Integration 3
3. Part 1 - run, Inside xv6 3
3.1. Implementation 4
3.2. Tests: echo, ls, and a Missing Program 4
3.3. Observations 5
4. Part 2 - Pingpong 5
4.1. Implementation 5
4.2. Execution and Verification 5
4.3. Observations 6
5. Verification and Deliverables 6
6. Conclusion 6

1. Introduction

The purpose of Laboratory Work No. 8 was to reproduce basic process-management and interprocess-communication operations **inside xv6**. In Laboratory Work No. 7 I used Linux system calls; in this laboratory I implemented **user/run.c** with **fork()**, **exec()**, and **wait()**, and **user/pingpong.c** using two pipes to exchange a byte between a parent and a child.

The experiments were performed in the Ubuntu virtual machine in **~/xv6-riscv**. xv6 was built using its Makefile and started through **QEMU**. The execution results in this report come from my recorded terminal session. The optional **primes.c** program was not part of the provided execution log, so it is not presented as completed.

2. Preparation and xv6 Build

First, I verified that the xv6 source directory contained the **Makefile**, **kernel**, and **user** directories. Running **make qemu** booted the system successfully and opened the xv6 shell. I then created **user/run.c**.

2.1. Makefile Integration

Every new xv6 user program must be listed in the **UPROGS** section of the Makefile. My first attempt accidentally sent the text **$U/_run** to the Ubuntu shell instead of editing the Makefile. As a result, the xv6 shell initially printed **exec run failed**: the program had not been included in the file-system image.

```
$ grep -n -E '_zombie|_run' Makefile
152:    $U/_zombie\

$ run                    # xv6, before Makefile correction
exec run failed
```

I corrected the Makefile with a short Python script, using the existing **_zombie** entry as the insertion point. The next verification showed that **_run** was registered. Later I added **_pingpong** in the same way.

```
$ grep -n -E '_zombie|_run|_pingpong' Makefile
152:    $U/_zombie\
153:    $U/_run\
154:    $U/_pingpong\
```

The subsequent **make qemu** build compiled both **user/run.c** and **user/pingpong.c**, linked **user/_run** and **user/_pingpong**, and included them in **fs.img**. The initial registration error was therefore resolved.

3. Part 1 - run, Inside xv6

The program **run.c** creates a child with **fork()**. In the child, **exec()** replaces the running program with **echo**; meanwhile, the parent calls **wait(&status)** and prints the status once the child has finished.

3.1. Implementation

```
int pid = fork();
if(pid < 0){
  printf("fork failed\n");
  exit(1);
}
if(pid == 0){
  printf("child: my pid is %d\n", getpid());
  char *args[] = { "echo", "hello", "from", "the", "child", 0 };
  exec("echo", args);
  printf("exec failed\n");
  exit(1);
}
int status;
wait(&status);
printf("parent: child %d ended with status %d\n", pid, status);
```

3.2. Tests: echo, ls, and a Missing Program

Once **_run** was added to the Makefile, I rebuilt xv6 and executed **run**. The first successful test showed parent PID 3, child PID 4, the message from **echo**, and exit status **0**.

```
$ run
parent: my pid is 3
child: my pid is 4
hello from the child
parent: child 4 ended with status 0
```

Next, I changed the argument array to **{ "ls", 0 }** and called **exec("ls", args)**. The program printed the xv6 directory listing, including names such as **README**, **echo**, **ls**, and **run**, and again finished with status 0.

```
$ run
parent: my pid is 3
child: my pid is 4
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 36728
echo           2 4 35592
...
ls             2 10 42880
run            2 23 35888
...
parent: child 4 ended with status 0
```

Finally, I used **exec("nosuchprog", args)**. Since this executable did not exist, **exec()** returned and the child printed the error message before exiting with status **1**.

```
$ run
parent: my pid is 3
child: my pid is 4
exec failed
parent: child 4 ended with status 1
```

After these tests I restored the source to the **echo** version. A **grep** check confirmed that **user/run.c** again contained the original argument array and **exec("echo", args)**.

3.3. Observations

1. **Which three system calls are used in xv6 and on Linux? What is the same?** The xv6 program uses **fork()**, **exec()**, and **wait()**. My Linux Lab 7 program used **fork()**, **execlp()**, and **waitpid()**. Both follow the same process lifecycle: create a child, replace the child's program image, and collect its termination status. The interfaces and available libraries differ, but the underlying operations are equivalent.

2. **Who prints "hello from the child": run or echo?** The **echo** executable prints the text. After **exec("echo", args)** succeeds, the child is no longer executing the original **run** program. This was confirmed by the captured message and status 0.

3. **Why is the failed-exec status 1 here but 127 on Linux?** In this xv6 program the failure branch ends with **exit(1)**. In my Lab 7 Linux program the corresponding branch ended with **exit(127)**. These are explicit exit codes selected by the programs, not a fundamental disagreement between the operating systems.

4. Part 2 - Pingpong

The **pingpong.c** program uses two pipes: **p2c** for the parent-to-child direction, and **c2p** for the child-to-parent direction. After **fork()**, each process closes the pipe ends it does not need. The parent writes one byte, the child reads it and prints **received ping**, then sends the byte back. The parent reads the response and prints **received pong**.

4.1. Implementation

```
int p2c[2], c2p[2];
char b = 'x';
pipe(p2c);
pipe(c2p);
if(fork() == 0){
  close(p2c[1]); close(c2p[0]);
  read(p2c[0], &b, 1);
  printf("%d: received ping\n", getpid());
  write(c2p[1], &b, 1);
  close(p2c[0]); close(c2p[1]);
  exit(0);
}
close(p2c[0]); close(c2p[1]);
write(p2c[1], &b, 1);
read(c2p[0], &b, 1);
printf("%d: received pong\n", getpid());
close(p2c[1]); close(c2p[0]);
wait(0);
```

The **read(c2p[0], &b, 1)** in the parent is a blocking operation: it cannot return with a byte until the child replies. The parent subsequently calls **wait(0)** to collect the child's termination status. The additional **close()** calls release unused and completed pipe endpoints.

4.2. Execution and Verification

After the Makefile was updated, **make qemu** successfully built **user/_pingpong**. I then ran the program three times from the xv6 shell. The captured outputs were:

```
$ pingpong
4: received ping
3: received pong
```

```
$ pingpong
6: received ping
5: received pong
$ pingpong
8: received ping
7: received pong
```

All three executions showed the required order: the child printed **ping** first, and the parent printed **pong** second. The process IDs changed for each new pair of processes, confirming that new parent-child executions were created.

4.3. Observations

1. **Why are TWO pipes needed instead of one?** Each pipe is a one-directional byte stream in this design. The first carries the byte from parent to child and the second carries the reply from child to parent. Using separate pipes provides two clearly separated communication directions and prevents both processes from reading and writing on the same stream in an ambiguous way.

2. **Which call makes the parent wait for the reply?** The parent blocks on **read(c2p[0], &b, 1)** until the child writes a byte into **c2p[1]**. The later **wait(0)** does not wait for the reply itself; it waits to collect the child's exit status.

5. Verification and Deliverables

The recorded commands verified that the source file **user/pingpong.c** existed, that the Makefile contained the **$U/_pingpong** entry, and that **pingpong.c** used **pipe()**, **fork()**, **read()**, **write()**, and **wait()**. The logs also demonstrated the successful build and execution of both required programs.

The terminal transcript showed that **notes.md** on the virtual machine contained the two Part 2 answers, but did not yet contain the Part 1 answers. For a complete submission, all five Observe responses are therefore included in this report; the corresponding **notes.md** should also be updated before pushing the work to the course repository. The optional **primes** task was not shown in the captured work.

```
Submission files required for the completed tasks:
  user/run.c
  user/pingpong.c
  Makefile (UPROGS updated)
  notes.md (Part 1 and Part 2 observations)
  Laboratory report
```

6. Conclusion

In this laboratory work I implemented process creation and execution inside xv6 and compared these mechanisms with Linux. The **run** experiments demonstrated **fork()**, **exec()**, and **wait()** with three cases: a successful **echo** execution, a successful **ls** execution, and an intentionally failed program lookup. The recorded exit statuses were **0**, **0**, and **1**, respectively.

I also implemented **pingpong**, where a parent and child communicated through two pipes. Three successful tests showed the same message order, **received ping** followed by **received pong**. The initial failure to start **run** was caused by a missing Makefile entry and was resolved by registering the program in **UPROGS** and rebuilding xv6. The completed exercises helped me understand how familiar Linux process and pipe mechanisms can be implemented and observed in a small teaching operating system.